

L & T EDUTECH

Theory Question Paper - Set B (Answers)

Course: Node.js & Asynchronous JavaScript

Semester: 5th Semester

Exam: CIA - 2

Name: Sri Ramanarayana S N

Roll Number: 2461029

Class: 5BTIT

Date: 27-08-2026

Q1. If app.js is not present in the current folder, Node.js reports an error similar to:

Error: Cannot find module 'C:\path\to\app.js'

Node.js cannot locate the file.

Fix: Move to the folder containing app.js using cd, or provide the correct file path:

node path/to/app.js

Q3. The official fs documentation provides:

- Available functions such as readFile(), readFileSync()
- Parameters and return values
- Callback/Promise usage
- Error behavior
- Examples

It helps choose between methods because synchronous methods block execution, while asynchronous methods allow other work to continue.

For example:

- `fs.readFile()` => Asynchronous
 - `fs.readFileSync()` => Synchronous
-

Q5. Given:

```
console.log(process.argv);
```

and `node app.js => hello world`

The array contains:

```
[  
  'path-to-node',  
  'path-to-app.js',  
  'hello',  
  'world'  
]
```

More precisely:

- `process.argv[0]` → path to Node.js executable
- `process.argv[1]` → path to `app.js`
- `process.argv[2]` → `"hello"`
- `process.argv[3]` → `"world"`

NOTE: The exact first two strings depend on the system's installation paths.

Q8. `const fs = require('fs');`

```
function readFileSync(path, callback) {  
  fs.readFile(path, 'utf8', (err, data) => {  
    callback(err, data);  
  });  
}
```

The **error-first convention** means:

```
callback(error, result);
```

If there is no error:

```
callback(null, data);
```

Q10. Given context's related code:

```
function delay(ms) {  
    return new Promise(resolve => {  
        setTimeout(resolve, ms);  
    });  
}
```

Example:

```
await delay(1000);
```

```
console.log("Done");
```

Here, The Promise resolves after approximately 1000 milliseconds.

Q11. Given:

```
async function f() {  
    try {  
        await Promise.reject('err1');  
    }  
    catch (e) {  
        console.log('caught:', e);  
    }  
}
```

```
f();
```

Output:

caught: err1

Q12. Given:

```
function a() {  
    console.log('a');  
    b();  
}
```

```
function b() {  
    console.log('b');  
}  
a();  
console.log('c');
```

Execution flow:

1. a() is called.
2. a is printed.
3. b() is called.
4. b is printed.
5. a() finishes.
6. c is printed.

Exact output:

a
b
c

Q13. The micro-task queue contains tasks such as:

- Promise callbacks
- queueMicrotask()

The macro-task / task queue contains tasks such as:

- Timers
- Some I/O callbacks
- setImmediate() in Node.js

After the current synchronous execution completes, the event loop drains the microtask queue before moving on to the next macro task.

Q14. Given:

```
function sumArray(arr, i = 0) {  
    if (i === arr.length) return 0;  
    return arr[i] + sumArray(arr, i + 1);  
}
```

Problem Identified:

Each recursive call is placed on the call stack. For a very large array, too many recursive calls are created, causing 'Maximum call stack size exceeded'

Concrete fix that is enclosed:

Use an iterative loop instead:

```
function sumArray(arr) {  
    let sum = 0;  
    for (let i = 0; i < arr.length; i++) {  
        sum += arr[i];  
    }  
}
```

```
        return sum;
    }
}
```

This avoids excessive call-stack usage.

Q15. Given:

math.js:

```
function add(a, b) {
    return a + b;
}

module.exports = add;
```

app.js:

```
const add = require('./math');
console.log(add(5, 3));
```

Output: 8

Practical Question Paper - Set 2 (Answers)

Course: Node.js & Asynchronous JavaScript

Semester: 5th Semester

Exam: CIA - 2

Name: Sri Ramanarayana S N

Roll Number: 2461029

Class: 5BTIT

Date: 27-08-2026

Task 1

Requirements:

1. Create a Node.js project.
2. Create countdown.js.
3. Run it.
4. Print "Countdown App Ready".

Step 1: Create folder

```
mkdir countdown-app
```

```
cd countdown-app
```

Step 2: Initialize Node.js

```
npm init -y
```

This creates a file named package.json

Step 3: Create countdown.js

```
console.log("Countdown App Ready");
```

Step 4: Run

```
node countdown.js
```

Output:

Countdown App Ready

Task 2**Requirements:**

Explain using a comment how V8 and libuv allow the application to continue executing while a timer runs.

In file countdown.js:

```
setTimeout(() => {  
    console.log("Timer finished");  
}, 3000);  
console.log("This message prints immediately");
```

Output:

This message prints immediately
Timer finished

Task 3

In file Basic countdown:

```
let seconds = 5;  
const timer = setInterval(() => {  
    console.log("Remaining:", seconds);  
    seconds--;  
    if (seconds < 0) {  
        clearInterval(timer);  
        console.log("Countdown complete!");  
    }  
}
```

```
}, 1000);
```

Output:

```
Remaining: 5  
Remaining: 4  
Remaining: 3  
Remaining: 2  
Remaining: 1  
Remaining: 0  
Countdown complete!
```

Task 5

Code:

```
const seconds = parseInt(process.argv[2]);  
if (isNaN(seconds) || seconds <= 0) {  
    console.log("Please provide a valid positive number.");  
    process.exit(1);  
}  
  
console.log(`Countdown started: ${seconds} seconds`);  
console.log("Type \"cancel\" and press Enter to stop.");  
let remaining = seconds;  
const timer = setInterval(() => {  
    console.log(`Remaining: ${remaining}`);  
    remaining--;  
    if (remaining < 0) {  
        clearInterval(timer);  
        console.log("Time's up!");  
    }  
}, 1000);
```

```

        process.exit(0);
    }
}, 1000);

process.stdin.setEncoding("utf8");
process.stdin.on("data", (input) => {
    if (input.trim().toLowerCase() === "cancel") {
        clearInterval(timer);
        console.log("Countdown cancelled.");
        process.exit(0);
    }
});

```

To Run:

```
node countdown.js 10
```

Task 8

Create:

```
checkTimeLeftCallback(seconds, callback)
```

Code:

```

function checkTimeLeftCallback(seconds, callback) {
    setTimeout(() => {
        const remaining = seconds;
        callback(null, remaining);
    }, 1000);
}

checkTimeLeftCallback(5, (err, remaining) => {

```

```
        if (err) {  
            console.error(err);  
            return;  
        }  
        console.log("Time remaining:", remaining);  
    });
```

Output:

Time remaining: 5

Error-first convention:

```
    callback(error, result);
```

So, if there is no error:

```
    callback(null, remaining);
```

Task 9

```
let remaining = 5;  
const interval = setInterval(() => {  
    console.log(`Remaining: ${remaining}`);  
    remaining--;  
    if (remaining < 0) {  
        clearInterval(interval);  
    }  
}, 1000);
```

```
setTimeout(() => {  
    console.log("Time's up!");  
}, 6000);
```

Output:

Remaining: 5
Remaining: 4
Remaining: 3
Remaining: 2
Remaining: 1
Remaining: 0
Time's up!

Task 10**Code:**

```
function checkTimeLeftPromise(seconds) {  
    return new Promise((resolve, reject) => {  
        if (seconds < 0) {  
            reject(new Error("Invalid duration"));  
            return;  
        }  
        setTimeout(() => {  
            resolve(seconds);  
        }, 1000);  
    });  
}
```

Using .then() and .catch():

```
checkTimeLeftPromise(5)  
    .then((remaining) => {  
        console.log("Time remaining:", remaining);  
    })
```

```
    })  
    .catch((error) => {  
        console.error("Error:", error.message);  
    });
```

Output

Time remaining: 5

Task 11

Code:

```
async function runCountdownAsync(seconds) {  
    try {  
        const remaining = await checkTimeLeftPromise(seconds);  
        console.log("Time remaining:", remaining);  
    }  
    catch (error) {  
        console.error("Caught error:", error.message);  
    }  
}  
  
runCountdownAsync(-5);
```

Output:

Caught error: Invalid duration

Task 13

Code:

```
console.log("Synchronous");  
  
Promise.resolve().then(() => {
```

```
        console.log("Promise");
    });
    setTimeout(() => {
        console.log("Timer");
    }, 0);
```

Output:

Synchronous

Promise

Timer

Task 14

Code:

```
const EventEmitter = require("events");
const countdownEmitter = new EventEmitter();
countdownEmitter.on("timeUp", () => {
    console.log("Notification: Time is up!");
});
let remaining = 3;
const timer = setInterval(() => {
    console.log(`Remaining: ${remaining}`);
    remaining--;
    if (remaining < 0) {
        clearInterval(timer);
        countdownEmitter.emit("timeUp");
    }
}, 1000);
```

Output:

Remaining: 3

Remaining: 2

Remaining: 1

Remaining: 0

Notification: Time is up!

Git Hub Link: https://github.com/SriRam-0511/L-T_Sem5.git
